

Chapitre 4 : Langage SQL - partie II

• WHERE

La commande WHERE dans une requête SQL permet d'extraire les lignes d'une base de données qui respectent une condition. Cela permet d'obtenir uniquement les informations désirées.

Syntaxe :

La commande WHERE s'utilise en complément à une requête utilisant SELECT. La façon la plus simple de l'utiliser est la suivante :

```
SELECT nom_colonnes
```

```
FROM nom_table
```

```
WHERE condition
```

Exemple :

Imaginons une base de données appelée « client » qui contient le nom des clients, le nombre de commandes qu'ils ont effectués et leur ville :

id	nom	nbr_commande	ville
1	Paul	3	paris
2	Maurice	0	rennes
3	Joséphine	1	toulouse
4	Gérard	7	paris

Pour obtenir seulement la liste des clients qui habitent à Paris, il faut effectuer la requête suivante :

```
SELECT *
```

```
FROM client
```

```
WHERE ville = 'paris'
```

Résultat :

id	nom	nbr_commande	ville
1	Paul	3	paris
4	Gérard	7	paris



Opérateurs de comparaison

La liste ci-jointe présente les opérateurs les plus couramment utilisés avec **WHERE** :

Opérateur	Description
=	Égale
<>	Pas égale
!=	Pas égale
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égale à
<=	Inférieur ou égale à
IN	Liste de plusieurs valeurs possibles
BETWEEN	Valeur comprise dans un intervalle donnée (utile pour les nombres ou dates)
LIKE	Recherche en spécifiant le début, milieu ou fin d'un mot.
IS NULL	Valeur est nulle
IS NOT NULL	Valeur n'est pas nulle

• AND - OR

Une requête SQL peut être restreinte à l'aide de la condition WHERE. Les opérateurs logiques AND et OR peuvent être utilisées au sein de la commande WHERE pour combiner des conditions.

Syntaxe :

Les opérateurs sont à ajoutés dans la condition WHERE. Ils peuvent être combinés à l'infini pour filtrer les données comme souhaités.

L'opérateur **AND** permet de s'assurer que la condition1 **ET** la condition2 sont vraies :

SELECT nom_colonnes

FROM nom_table

WHERE condition1 **AND** condition2

L'opérateur **OR** vérifie quant à lui que la condition1 **OU** la condition2 est vrai :

SELECT nom_colonnes

FROM nom_table

WHERE condition1 **OR** condition2

**IN**

L'opérateur logique IN dans SQL s'utilise avec la commande WHERE pour vérifier si une colonne est égale à une des valeurs comprises dans un set de valeurs déterminées. C'est une méthode simple pour vérifier si une colonne est égale à une valeur OU une autre valeur OU une autre valeur et ainsi de suite, sans avoir à utiliser de multiples fois l'opérateur OR.

Syntaxe :

Pour chercher toutes les lignes où la colonne « nom_colonne » est égale à 'valeur 1' OU 'valeur 2' ou 'valeur 3', il est possible d'utiliser la syntaxe suivante :

```
SELECT nom_colonne
FROM table
WHERE nom_colonne IN ( valeur1, valeur2, valeur3, ... )
```

La syntaxe utilisée avec l'opérateur est plus simple que d'utiliser une succession d'opérateur OR. Pour le montrer concrètement avec un exemple, voici 2 requêtes qui retourneront les mêmes résultats, l'une utilise l'opérateur IN, tandis que l'autre utilise plusieurs OR.

Requête avec plusieurs OR :

```
SELECT prenom
FROM utilisateur
WHERE prenom = 'Maurice' OR prenom = 'Marie' OR prenom = 'Thimoté'
```

Requête équivalent avec l'opérateur IN:

```
SELECT prenom
FROM utilisateur
WHERE prenom IN ( 'Maurice', 'Marie', 'Thimoté' )
```

• BETWEEN

L'opérateur BETWEEN est utilisé dans une requête SQL pour sélectionner un **intervalle** de données dans une requête utilisant WHERE. L'intervalle peut être constitué de chaînes de caractères, de nombres ou de dates. L'exemple le plus concret consiste par exemple à récupérer uniquement les enregistrements entre 2 dates définies.

Syntaxe :

L'utilisation de la commande BETWEEN s'effectue de la manière suivante :

```
SELECT *
FROM table
WHERE nom_colonne BETWEEN 'valeur1' AND 'valeur2'
```



La requête retourne toutes les lignes dont la valeur de la colonne « nom_colonne » sera comprise entre **valeur1** et **valeur2**.

Remarque : l'opérateur **NOT BETWEEN** donne un résultat contraire au résultat donné par l'opérateur **BETWEEN**.

• LIKE

L'opérateur **LIKE** est utilisé dans la clause WHERE des requêtes SQL. Ce mot-clé permet d'effectuer une recherche sur un modèle particulier. Il est par exemple possible de rechercher les enregistrements dont la valeur d'une colonne commence par telle ou telle lettre. Les modèles de recherche sont multiples.

Syntaxe :

La syntaxe à utiliser pour utiliser l'opérateur LIKE est la suivante :

```
SELECT *
FROM table
WHERE colonne LIKE modele
```

Dans cet exemple le « modèle » n'a pas été défini, mais il ressemble très généralement à l'un des exemples suivants :

- **LIKE 'a'** : le caractère « % » est un caractère qui remplace tous les autres caractères. Ainsi, ce modèle permet de rechercher toutes les chaînes de caractère qui se terminent par un « a ».
- **LIKE 'a%'** : ce modèle permet de rechercher toutes les lignes de « colonne » qui commencent par un « a ».
- **LIKE '%a%'** : ce modèle est utilisé pour rechercher tous les enregistrements qui utilisent le caractère « a ».
- **LIKE 'pa%on'** : ce modèle permet de rechercher les chaînes qui commencent par « pa » et qui se terminent par « on », comme « pantalon » ou « pardon ».
- **LIKE 'a_c'** : peu utilisé, le caractère « _ » (underscore) peut être remplacé par n'importe quel caractère, mais un seul caractère uniquement (alors que le symbole pourcentage « % » peut être remplacé par un nombre incalculable de caractères).

• ALIAS « AS »

Dans le langage SQL il est possible d'utiliser des alias pour renommer temporairement une colonne dans une requête. Cette astuce est particulièrement utile pour faciliter la lecture des requêtes.

Syntaxe :

```
SELECT id AS identifiant
FROM nom_table
```



ORDER BY

La commande ORDER BY permet de trier les lignes dans un résultat d'une requête SQL. Il est possible de trier les données par ordre ascendant ou descendant.

Syntaxe :

Une requête où l'on souhaite filtrer l'ordre des résultats utilise la commande ORDER BY de la sorte :

```
SELECT colonne1, colonne2
```

```
FROM table
```

```
ORDER BY colonne1
```

Par défaut les résultats sont classés par ordre ascendant, toutefois il est possible d'inverser l'ordre en utilisant le suffixe DESC après le nom de la colonne.

```
SELECT colonne1, colonne2
```

```
FROM table
```

```
ORDER BY colonne1 DESC
```

• Fonctions de statistiques

Il existe plusieurs fonctions qui peuvent être utilisées pour manipuler plusieurs enregistrements, il s'agit des fonctions d'agrégations statistiques, les principales sont les suivantes :

- **AVG()** pour calculer la moyenne d'un set de valeur.
- **COUNT()** pour compter le nombre de lignes concernées.
- **MAX()** pour récupérer la plus haute valeur.
- **MIN()** pour récupérer la plus petite valeur.
- **SUM()** pour calculer la somme de plusieurs lignes.

• UPDATE

La commande UPDATE permet d'effectuer des modifications sur des lignes existantes. Très souvent cette commande est utilisée avec WHERE pour spécifier sur quelles lignes doivent porter la ou les modifications.

Syntaxe :

La syntaxe basique d'une requête utilisant UPDATE est la suivante :

```
UPDATE table
```

```
SET nom_colonne_1 = 'nouvelle valeur'
```

```
WHERE condition
```



La syntaxe permet d'attribuer une nouvelle valeur à la colonne **nom_colonne_1** pour les lignes qui respectent la condition stipulée avec **WHERE**. Il est aussi possible d'attribuer la même valeur à la colonne **nom_colonne_1** pour toutes les lignes d'une table si la condition **WHERE** n'était pas utilisée.

A noter, pour spécifier en une seule fois plusieurs modification, il faut séparer les attributions de valeur par des virgules. Ainsi la syntaxe deviendrait la suivante :

UPDATE table

SET colonne_1 = 'valeur 1', colonne_2 = 'valeur 2', colonne_3 = 'valeur 3'

WHERE condition

- **DELETE**

La commande DELETE en SQL permet de supprimer des lignes dans une table. En utilisant cette commande associée à WHERE il est possible de sélectionner les lignes concernées qui seront supprimées.

Syntaxe :

La syntaxe pour supprimer des lignes est la suivante :

DELETE FROM table

WHERE condition

Attention : s'il n'y a pas de condition WHERE alors toutes les lignes seront supprimées et la table sera alors vide.

- **IS NULL / IS NOT NULL**

Dans le langage SQL, l'opérateur IS permet de filtrer les résultats qui contiennent la valeur NULL. Cet opérateur est indispensable car la valeur NULL est une valeur inconnue et ne peut par conséquent pas être filtrée par les opérateurs de comparaison (cf. égal, inférieur, supérieur ou différent).

Syntaxe :

Pour filtrer les résultats où les champs d'une colonne sont à NULL il convient d'utiliser la syntaxe suivante :

SELECT *

FROM `table`

WHERE nom_colonne **IS NULL**

A l'inverse, pour filtrer les résultats et obtenir uniquement les enregistrements qui ne sont pas NULL, il convient d'utiliser la syntaxe suivante :



```
SELECT *  
FROM `table`
```

WHERE nom_colonne **IS NOT NULL**

- **DISTINCT**

L'utilisation de la commande **SELECT** en SQL permet de lire toutes les données d'une ou plusieurs colonnes. Cette commande peut potentiellement afficher des lignes en doubles. Pour éviter des redondances dans les résultats il faut simplement ajouter **DISTINCT** après le mot **SELECT**.

Syntaxe :

L'utilisation basique de cette commande consiste alors à effectuer la requête suivante :

```
SELECT DISTINCT ma_colonne  
FROM nom_du_tableau
```

- **LIMIT**

La clause **LIMIT** est à utiliser dans une requête SQL pour spécifier le nombre maximum de résultats que l'on souhaite obtenir. Cette clause est souvent associée à un **OFFSET**, c'est-à-dire effectuer un décalage sur le jeu de résultat. Ces 2 clauses permettent par exemple d'effectuer un système de pagination.

Syntaxe:

```
SELECT *  
FROM table  
LIMIT 10
```

Cette requête permet de récupérer seulement les 10 premiers résultats d'une table. Bien entendu, si la table contient moins de 10 résultats, alors la requête retournera toutes les lignes.

Bon à savoir : la bonne pratique lorsque l'on utilise **LIMIT** consiste à utiliser également la clause **ORDER BY** pour s'assurer que quoi qu'il en soit ce sont toujours les bonnes données qui sont présentées. En effet, si le système de tri est non spécifié, alors il est en principe inconnu et les résultats peuvent être imprévisibles.

LIMIT ET OFFSET

L'offset est une méthode simple de décaler les lignes à obtenir. La syntaxe pour utiliser une limite et un offset est la suivante :

```
SELECT *  
FROM table  
LIMIT 10 OFFSET 5
```

Cette requête permet de récupérer les résultats 6 à 15 (car l'**OFFSET** commence toujours à 0).



GROUP BY

La commande GROUP BY est utilisée en SQL pour grouper plusieurs résultats et utiliser une fonction sur un groupe de résultat. Sur une table qui contient toutes les ventes d'un magasin, il est par exemple possible de regrouper les ventes par client et d'obtenir le coût total des achats pour chaque client.

Syntaxe :

De façon générale, la commande GROUP BY s'utilise de la façon suivante :

```
SELECT colonne1, fonction(colonne2)
```

```
FROM table
```

```
GROUP BY colonne1
```

A noter : GROUP BY doit toujours s'utiliser après la commande WHERE et avant la commande HAVING.

Exemple d'utilisation de GROUP BY :

Prenons en considération une table « achat » qui résume les ventes d'une boutique :

id	client	tarif	date
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Ce tableau contient une colonne qui sert d'identifiant pour chaque ligne, une autre qui contient le nom du client, le coût de la vente et la date d'achat.

Pour obtenir le coût total de chaque client en regroupant les commandes des mêmes clients, il faut utiliser la requête suivante :

```
SELECT client, SUM(tarif)
```

```
FROM achat
```

```
GROUP BY client
```

La fonction **SUM()** permet d'additionner la valeur de chaque tarif pour un même client. Le résultat sera donc le suivant :

client	SUM(tarif)
Pierre	262
Simon	47
Marie	38



HAVING

La condition HAVING en SQL est presque similaire à WHERE à la seule différence que HAVING permet de filtrer en utilisant des fonctions telles que SUM(), COUNT(), AVG(), MIN() ou MAX().

Syntaxe:

L'utilisation de HAVING s'utilise de la manière suivante :

SELECT colonne1, SUM(colonne2)

FROM nom_table

GROUP BY colonne1

HAVING fonction(colonne2) **opérateur** valeur

Cela permet donc de SÉLECTIONNER les colonnes DE la table « nom_table » en GROUPANT les lignes qui ont des valeurs identiques sur la colonne « colonne1 » et que la condition de HAVING soit respectée.

Exemple :

Pour utiliser un exemple concret, imaginons une table « achat » qui contient les achats de différents clients avec le coût du panier pour chaque achat.

id	client	tarif	date
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Si dans cette table on souhaite récupérer la liste des clients qui ont commandé plus de 40, alors il est possible d'utiliser la requête suivante :

SELECT client, SUM(tarif)

FROM achat

GROUP BY client

HAVING SUM(tarif) > 40

Résultat :

client	SUM(tarif)
Pierre	162
Simon	47

La cliente « Marie » a cumulé 38 d'achat (un achat de 18 et un autre de 20) ce qui est inférieur à la limite de 40€ imposée par HAVING. En conséquent cette ligne n'est pas affichée dans le résultat.